

Programación concurrente y paralela

Práctica 1 - Introducción

Primer Cuatrimestre 2026

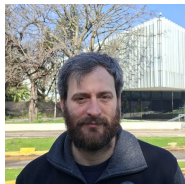


1. Presentación de la materia
2. Semántica, diagrama de transición de estados
3. Entorno Java

Pablo Terlisky

JTP

terlisky [[at]] dc.uba.ar



Tomás Chimenti

JTP

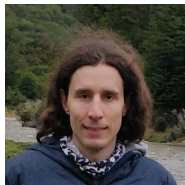
tach.365 [[at]] gmail.com



Jorge Szabo

AY1

jorgecszabo [[at]] gmail.com



Julián Zylber

AY1

jzylber [[at]] dc.uba.ar



- Teóricas: Lunes 17-22 - Aula 3 (pb. 1)
- Práctica: Miércoles 17-22 - Aula 1207 / Laboratorio 1106 (pb. $0+\infty$)

El calendario está en el Campus de la materia. Prestenle atención ya que puede sufrir cambios.

- Clases teóricas los lunes y prácticas los miércoles
- Las clases prácticas serán práctica-laboratorio
- 2da mitad de la materia aún más: teórico-prácticas-laboratorio

La práctica tiene un servidor de Discord



<https://discord.gg/gYSUk6vyU>



- 2 parciales
 - Aún estamos planificando la modalidad
- 1 trabajo práctico
 - grupal (2 personas)
 - Defensa oral
- 1 examen final (no hay promoción)

Semántica

- Denotamos un estado σ de la memoria por medio de variables y sus valores ($\{x = 1, y = 4, \text{var} = 3, \dots\}$)

- Función parcial que transforma estados en estados
- Podemos escribir el estado resultante con el operador de actualización de estado.
- Ejemplo: $P \stackrel{\text{def}}{=} x := 7$, luego $\llbracket P \rrbracket = \lambda\sigma.\sigma[x \mapsto 7]$
(dado el estado σ , P obtiene el resultado de cambiar el valor de x por 7 en σ)
- Ejemplo: $P \stackrel{\text{def}}{=} x := x + 1$
 $\llbracket P \rrbracket = \lambda\sigma.\sigma[x \mapsto \sigma x + 1]$
(dado el estado σ , P obtiene el resultado de tomar el valor de x en σ y sumarle 1)
- Nota: El operador de actualización de estado preserva las variables no mencionadas. Es decir, $(\llbracket P \rrbracket \sigma) y = \sigma y$.

Si tenemos $P \stackrel{\text{def}}{=}$

```
1 y := 0 ;  
2 while (x>0) {  
3     y := y + x ;  
4     x := x - 1 ;  
5 }
```

La semántica depende del valor inicial de x . Luego, el estado final se escribe como la expresión booleana `if then else`.

$$\llbracket P \rrbracket = \lambda\sigma. \text{if } \sigma x > 0 \text{ then } \sigma[x \mapsto 0, y \mapsto (\sigma x * \sigma x)/2] \text{ else } \sigma[y \mapsto 0]$$

¿Pero qué pasa si para alguna entrada el programa no termina?

Si tenemos $P \stackrel{\text{def}}{=}$

```
1 y := 0 ;  
2 while (x!=0) {  
3     y := y + x ;  
4     x := x - 1 ;  
5 }
```

Un posible resultado de ejecutar P es que este no termine (Lo notamos como resultado *undefined*, o $\perp$)

$$\llbracket P \rrbracket = \lambda \sigma. \text{if } \sigma x \geq 0 \text{ then } s[x \mapsto 0, y \mapsto (\sigma x * (\sigma x + 1))/2] \text{ else } \perp$$

Consideremos

$$P \stackrel{\text{def}}{=}$$

P1 $x := 4$;
P2 $y := 2$

$$Q \stackrel{\text{def}}{=}$$

Q1 $y := 3$;
Q2 $x := 5$

Tenemos

$$\llbracket P \rrbracket = \lambda\sigma.\sigma[x \mapsto 4, y \mapsto 2]$$

$$\llbracket Q \rrbracket = \lambda\sigma.\sigma[x \mapsto 5, y \mapsto 3]$$

Semántica de correr concurrentemente P y Q ($\llbracket P|Q \rrbracket$):

$$\llbracket P|Q \rrbracket = \lambda\sigma.\sigma[x \mapsto 5, y \mapsto 3] \oplus \lambda\sigma.\sigma[x \mapsto 4, y \mapsto 2] \oplus \lambda\sigma.\sigma[x \mapsto 5, y \mapsto 2]$$

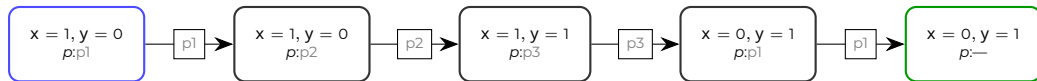
Diagrama de Transición

- Vamos a hacer el seguimiento de programas concurrentes escritos en pseudocódigo.
- Etiquetamos cada línea que consideramos atómica para nuestro análisis
 - Nota: podrían faltarnos escenarios y nuestra elección no es lo suficientemente desagregada
- Consideramos cada estado posible, marcando:
 - las variables del programa (globales y locales)
 - la etiqueta de la siguiente instrucción a ejecutar de cada thread (notamos — cuando ya no quedan instrucciones a ejecutar)

- Para cada estado, hay tantas transiciones salientes de ese estado como etiquetas que podrían ser ejecutadas.
- Los estados finales son cuando las siguientes instrucciones para todos los threads son —.

Ejemplo:

```
1 global x = 1
2 global y = 0
3
4 thread p
5 p1: while (x>0)
6 p2:   y := y + x
7 p3:   x := x - 1
```



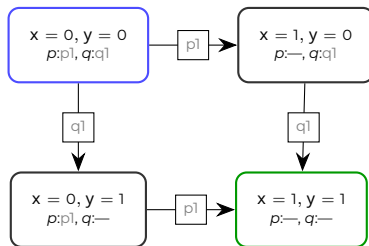
Notar que podrían haber diagramas infinitos

Cuando tenemos múltiples threads, el camino efectivo entre el estado inicial y uno final será una elección no determinística entre las posibilidades.

Tomemos el siguiente pseudocódigo:

```
1 global x
2
3 thread p
4 p1 x = 1
5
6 thread q
7 q1 y = 1
```

¿Cómo es el diagrama de transición?

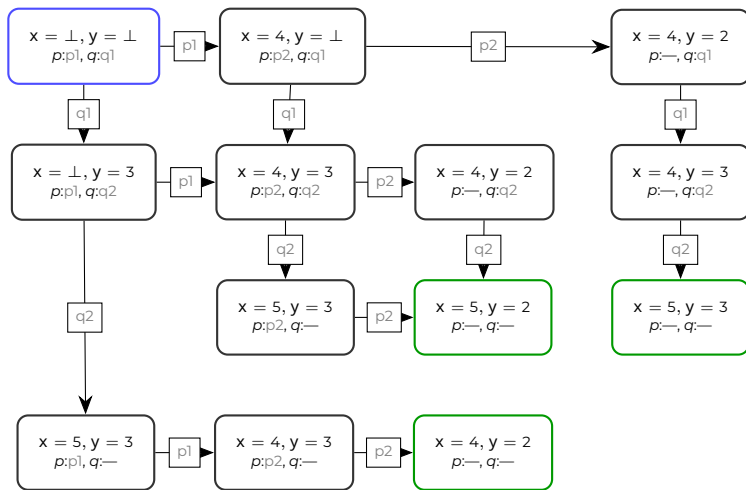


Existen dos ejecuciones posibles (y no tenemos control de cuál se dará), pero el estado final es el mismo.

Consideremos el caso de antes:

```
1 global x
2 global y
3
4 thread p
5 p1 x = 4
6 p2 y = 2
7
8 thread q
9 q1 y = 3
10 q2 x = 5
```

En este caso, sí existen múltiples estados finales. ¿Cómo queda el diagrama de transición?



Una *traza* de nuestro programa es la descripción de un camino dentro del diagrama de estados. Una forma de escribirla es como una tabla donde ponemos la línea de código ejecutada y los cambios que produce en el estado y la próxima instrucción

p	q	Estado
$x:=4$		$x=4;p:p2$
	$y:=3$	$y=3;q:q2$
	$x:=5$	$x=5;q:—$
$y:=2$		$y=2;p:—$

p	q	Estado	IP
$x:=4$		$x=4$	$p:p2$
$y:=2$		$y=2$	$p:—$
	$y:=3$	$y=3$	$q:q2$
	$y:=5$	$y=5$	$q:—$

Exclusión Mutua

Ejecutar múltiples threads con memoria compartida es que existen trazas que producen resultados indeseados.

Caso emblemático: Pérdida de sumas

```
1 global x = 0
2
3 thread p
4 p1:  x = x + 1
5
6 thread q
7 q1:  x = x + 1
```

A priori asumir que p1 y q1 son atómicas no refleja las posibilidades de nuestro sistema.

Expandimos el código separando la lectura de la variable compartida de su escritura

```
1 global x = 0
2
3 thread p
4 tmp
5 p1: tmp = x
6 p2: x = tmp + 1
7
8 thread q
9 tmp
10 q1: tmp = x
11 q2: x = tmp + 1
```

- La primera línea de cada thread declara la variable local `tmp`. En la práctica es un espacio de memoria exclusivo o un registro de la CPU.
- Podemos asumir atomicidad cuando operamos sobre variables locales, ya que los interleavings no hacen diferencia.
- Para describirlas en trazas podemos usar `p.tmp` y `q.tmp` (son variables independientes)

Podemos denotar la traza que pierde un incremento

p	q	Estado
$tmp = x$ $x = tmp + 1$	$tmp = x$ $x = tmp + 1$	$p.tmp=0;p:p2$ $q.tmp=0;q:q2$ $x=1;p:—$ $x=1;p:—$

- **Sección crítica:** La porción del código de nuestro thread que debería ejecutarse de a lo sumo un thread concurrentemente.
- **Problema de la Exclusión Mutua:** El de garantizar que se cumple este requerimiento sobre las secciones críticas.

Cuando analizamos si un programa resuelve el problema de la exclusión mutua, asumimos que:

- La sección no crítica no manipula las variables de la sección crítica
- La sección crítica siempre termina (No se asume para la sección no crítica)
- El scheduler no ignorará a un thread por siempre (Es *weakly fair*)

Para ver que efectivamente se resuelva, tenemos que certificar que se cumplan las propiedades:

- Mutex
- Ausencia de Deadlocks/Livelocks
- Garantía de Entrada

¿Resuelve este programa el problema de la exclusión mutua?

```
1 global espera = 0
2
3 thread p
4 p1: //SNC
5 p2: espera = espera + 1
6 p3: while (espera > 1)
7 p4: //SC
8 p5: espera = espera - 1
9 p6: //SNC
10
11 thread q
12 q1: //SNC
13 q2: espera = espera + 1
14 q3: while (espera > 1)
15 q4: //SC
16 q5: espera = espera - 1
17 q6: //SNC
```

El programa **no resuelve** el problema de la exclusión mutua. En particular, no cumple **ninguna** de las tres propiedades. Esta es una traza que genera un deadlock:

p	q	Estado
<code>espera = espera + 1</code> <code>while (espera > 1)</code> <code>while (espera > 1)</code>	<code>espera = espera + 1</code> <code>while (espera > 1)</code> <code>while (espera > 1)</code>	<code>espera = 1;p:p3</code> <code>espera = 2;q:q3</code> <code>p:p3</code> <code>p:p3</code> <code>q:q3</code> <code>q:q3</code>

El ejemplo no cumple garantía de entrada, la existencia de un deadlock lo prueba.

Si bien dejamos en la traza instrucciones que son atómicas, dado que no necesitamos que nuestro interleaving cambie de contexto durante las mismas, nos sirve.

Para ver que el ejemplo no cumple mutex, debemos perder una suma como vimos anteriormente. Expandimos el código en los incrementos.

```
1 global espera = 0
2
3 thread p
4 lesp
5 p1: //SNC
6 p2: lesp = espera
7 p3: espera = lesp + 1
8 p4: while (espera > 1)
9 p5: //SC
10 p6: espera = espera - 1
11 p7: //SNC
```

```
10
11 thread q
12 lesp
13 q1: //SNC
14 q2: lesp = espera
15 q3: espera = lesp + 1
16 q4: while (espera > 1)
17 q5: //SC
18 q6: espera = espera - 1
19 q7: //SNC
```

Luego, tenemos una traza donde no se cumple la propiedad de Mutex:

<i>p</i>	<i>q</i>	Estado
//SNC lesp = espera		p:p2 p.lesp = 0;p:p3
	//SNC lesp = espera	q:q2 q.lesp = 0;q:q3
espera = lesp + 1		espera = 1;p:p4
	espera = lesp + 1	espera = 1;q:q4
while(espera>1)		p:p5
//SC		p:p6
	while(espera>1)	q:q5
	//SC	q:q6

A lo largo del tiempo se fueron incorporando operaciones a los sets de instrucciones de los procesadores que ejecutan atómicamente.

En este caso, ¿Cambia la situación de alguna de las propiedades si asumimos que podemos incrementar una variable atómicamente?

Ciertamente, **no cambia** para la ausencia de Deadlock, la traza que mostramos ya tiene las instrucciones de los incrementos ejecutando de corrido.

Sin embargo, ahora **sí se cumple** la propiedad de Mutex.

No basta con proveer una traza donde valga la propiedad, tenemos que mostrar que **ninguna traza posible la contradice**.

Una posibilidad es hacer el diagrama de transición completo, pero para programas medianos ya se vuelve impracticable.

En cambio, podemos argumentar (no vamos a pedir demasiado rigor formal) alguna invariante que nos permita deducir que la propiedad vale siempre.

En este ejemplo, podemos argumentar lo siguiente:

- Ambos threads incrementan `espera` antes de entrar en la S.C. y no la decrementan hasta haber salido.
- Siendo los incrementos atómicos, una vez ejecutadas ambos incrementos el valor de `espera` necesariamente es 2.
- Si los dos threads están en la S.C., en la traza, alguno de los dos es el segundo en salir del `while`. Supongamos el momento en que lo está evaluando.
- Ese thread ya incrementó (la instrucción precede al `while`) y el otro thread también, pues ya está en su S.C. (y aún no sale)
- Pero entonces `espera` vale 2, no es posible que el segundo thread salga del `while` y entre en la S.C.

Algoritmos conocidos que resuelven el problema de la exclusión mutua:

- **Dekker** (para 2 threads)
- **Peterson** (para 2 threads)
- **Bakery** (para N threads, N fijado de antemano)

Estos no requieren de instrucciones atómicas adicionales a la lectura y escritura de variables. Cuando se incorporan instrucciones atómicas como test-and-set, exchange, compare-and-swap y fetch-and-add, el problema de exclusión mutua puede resolverse de manera mucho más sencilla.

Implementación

Un proceso es un **contenedor de recursos**. Los threads que corren adentro son quienes efectivamente **ejecutan**.

Compartido por todos los threads

Espacio de direcciones, archivos abiertos, señales, variables de entorno

Propio de cada thread

Stack, registros/PC, estado de ejecución

Lo importante

Comunicar dos **procesos** requiere mecanismos explícitos del SO (pipes, sockets, memoria compartida). Comunicar dos **threads** del mismo proceso es trivial: ya comparten memoria, alcanza con leer y escribir la misma variable.

Esa memoria compartida es la gran ventaja de los threads y la fuente de los problemas que vamos a estudiar: **race conditions**, exclusión mutua, sincronización.

En los sistemas operativos modernos, la unidad que el scheduler despacha a un core es el **thread**, no el proceso. Cada core corre, en un instante dado, un thread a la vez (o unos pocos, con hyperthreading).

- **Crear un proceso** (fork) es costoso: hay que duplicar las tablas de páginas del padre.
- **Crear un thread** es barato: reusa el espacio de direcciones del proceso, sólo arma un stack nuevo.
- Cambiar de thread **dentro** del mismo proceso (*context switch*) es más liviano que cambiar de proceso: no hace falta cambiar la tabla de páginas.

Por eso, para paralelizar trabajo dentro de una misma aplicación se suelen preferir threads a procesos, salvo que haga falta aislar fallos o el lenguaje no dé paralelismo real con threads.

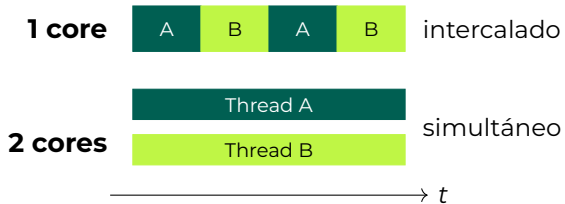
Concurrencia

Varios threads **en progreso** durante un mismo intervalo de tiempo, sin que eso implique que corran en el mismo instante.

Paralelismo

Varios threads ejecutando **literalmente al mismo instante**, en cores físicos distintos.

Con **un solo core** sólo hay concurrencia (una ilusión, lograda a fuerza de *time slicing*). Con **N cores**, hasta N threads pueden ser paralelos de verdad.



Scheduler

El componente del SO que decide, en cada core, qué thread (de entre los que están **listos** para ejecutar) corre a continuación, y durante cuánto tiempo (el *quantum*).

- Si hay **más** threads listos que cores disponibles, el scheduler los va turnando en cada core (time slicing).
- Si hay **tantos o menos** threads que cores, cada uno consigue su propio core, y ahí sí hay paralelismo real.
- Cada core suele tener su propia cola de listos, y el scheduler **balancea** threads entre colas para que ningún core quede ocioso mientras otro está sobrecargado.

- Cuando un thread se ejecuta de forma continua en un core, sus datos y estructuras de trabajo más frecuentes se cargan en las cachés locales (L1/L2).
- Mover un thread de un core a otro (migración de núcleo) introduce una penalización de rendimiento debido a la pérdida de afinidad de caché.

Afinidad (CPU affinity)

Preferencia (o restricción explícita) de que un thread corra siempre en el mismo core, o en un subconjunto fijo de cores, para conservar esa localidad de cache.

En Linux se puede pedir con `taskset` desde la terminal, o con `pthread_setaffinity_np` desde el código.

Implementar colas de listos independientes por cada núcleo introduce un nuevo desafío: el desbalanceo de carga. Puede ocurrir que un núcleo acumule varios hilos en espera mientras otro permanece completamente ocioso.

Para mitigar esto, el scheduler utiliza mecanismos de balanceo de carga (load balancing) con el fin de redistribuir los hilos y optimizar la utilización del procesador:

- **Work Stealing / Idle Balancing:** un core se queda sin nada para correr, y “roba” un thread de la cola de otro core (*work stealing*).
- **Balanceo periódico:** cada cierto intervalo, el scheduler compara la carga entre cores y migra threads si la diferencia es lo bastante grande.

El costo escondido

Todo mecanismo de balanceo de carga implica, por definición, una migración. Un scheduler muy agresivo que balancee de forma constante destruirá la localidad de caché lograda por los hilos.

Por esta razón, las políticas de balanceo reales no buscan una equidad perfecta en tiempo real. En su lugar, operan bajo criterios estrictos:

- Umbral de activación: Solo se migran hilos si la disparidad de carga entre los núcleos supera un margen preestablecido.
- Se aplican mecanismos de control para evitar oscilaciones rápidas (un efecto de ping-pong), impidiendo que un hilo sea devuelto a su núcleo original de inmediato y estabilizando así las decisiones del scheduler.

Linux (y la mayoría de los SO modernos) agrupa los cores en **dominios** jerárquicos, de más cercano a más lejano:

- Hilos SMT del mismo core físico.
- Cores que comparten cache L3 (mismo socket/chip).
- Cores de otro socket, en otro **nodo NUMA** (memoria principal más lejos, más lenta de alcanzar).

El balanceador prefiere migrar dentro del dominio más cercano (barato). Sólo cruza a uno más lejano si el desbalance es grande y persiste en el tiempo.

Kernel thread

Una entidad que el **sistema operativo** conoce y planifica directamente: aparece en la cola de listos del scheduler, y se le puede asignar un core.

User-level thread

Una entidad manejada enteramente por una **librería o runtime**, en espacio de usuario. El kernel no sabe que existe: sólo ve al proceso (y a los kernel threads que lo respaldan).

Un “modelo de threading” es, exactamente, **cómo se mapean** los threads que ve el programador contra los threads que planifica el kernel.

La diferencia **no** es qué instrucciones ejecuta cada uno: es **quién sabe que existe, y quién lo planifica**.

- Un **kernel thread** vive **dentro** del sistema operativo: tiene su propia entrada en las estructuras internas del kernel, y es el kernel quien decide cuándo corre y en qué core.
- Un **thread de usuario** es sólo **datos** guardados en la memoria de tu proceso: un stack, y un lugar donde guardar los registros cuando no está corriendo. El kernel no lo ve.

Mapeo (binding) de un thread de usuario a un kernel thread

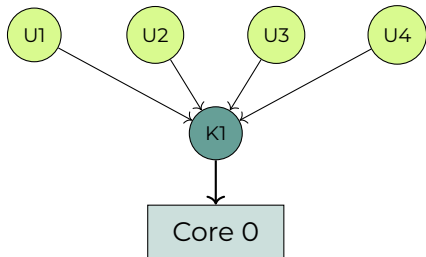
Que un thread de usuario esté mapeado a un kernel thread significa que, en ese instante, el kernel thread está ejecutando físicamente las instrucciones de ese thread de usuario: su stack, su program counter y sus registros son, por ahora, los de ese thread.

- El runtime guarda, por cada thread de usuario, su propio **contexto** (stack, registros, PC), igual que el kernel guarda uno por cada kernel thread.
- **Montar** un thread de usuario en un kernel thread es cargar ese contexto guardado en los registros reales. **Desmontar** es guardarlo de nuevo y dejar el kernel thread libre para otro.

Todos los threads de un proceso (N) se mapean sobre un **único** kernel thread. El scheduling entre ellos lo hace un scheduler en **espacio de usuario** (el runtime del lenguaje), no el sistema operativo.

- + Crear y cambiar de contexto entre threads es **muy barato** (no hace falta entrar al kernel).
- **Sin paralelismo real**: al haber un solo kernel thread, sólo uno de ellos puede correr a la vez, en un solo core.
- Si un thread hace una **syscall bloqueante**, el kernel sólo ve un hilo, y bloquea a **todos** los demás con él.

Nombre histórico: *green threads* (Java 1.1, primeras versiones de Ruby).

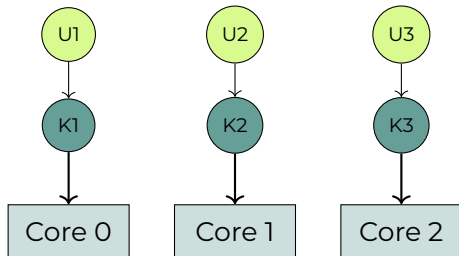


N threads de usuario, 1 kernel thread: por más threads que haya, sólo hay un core en juego a la vez.

Cada thread de usuario se mapea directamente contra **su propio** kernel thread. El scheduling lo hace enteramente el sistema operativo.

- + **Paralelismo real**: cada thread puede terminar en un core distinto.
- + Una syscall bloqueante en un thread **no afecta** a los demás.
- Crear un thread y cambiar de contexto son **más caros**: hay que pasar por el kernel.

Es el modelo de los **pthread**s en Linux, y el de `java.lang.Thread` tradicional (los llamados *platform threads*) en Java.



Cada thread de usuario tiene su propio kernel thread: los tres pueden estar en tres cores distintos, al mismo tiempo.

Un kernel thread **no es gratis**: el sistema operativo le reserva recursos reales (su propia estructura interna, su propio stack de kernel). Por eso todo sistema impone un **límite** a cuántos puede haber, por proceso y en total.

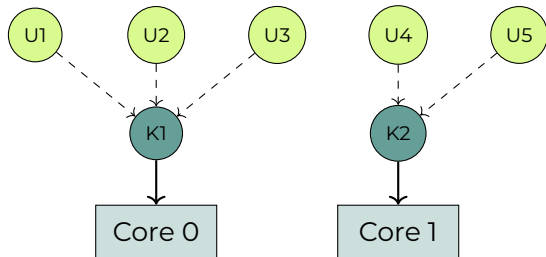
Tener más kernel threads que cores **no es un problema en sí mismo**, eso ya lo resuelve el scheduler turnándolos en cada core (time slicing, como vimos antes). El límite del que hablamos acá es otro: una cota **dura** sobre cuántos kernel threads pueden existir creados a la vez, corran o no en ese instante.

N threads de usuario se multiplexan sobre M kernel threads (típicamente $M \approx$ cantidad de cores). Un scheduler en espacio de usuario decide, en cada momento, qué thread de usuario ocupa cada kernel thread disponible.

Lo mejor de los dos mundos, a cierto costo

Threads baratos de crear (como N:1) **y** paralelismo real (como 1:1), porque hay $M \geq 2$ kernel threads de fondo. A cambio, el runtime necesita su propio scheduler, y tiene que resolver qué hacer cuando un thread de usuario hace una syscall bloqueante, para no bloquear a los demás con él.

Ejemplos: goroutines de Go, procesos de Erlang, threads de Solaris (donde nació el modelo).



Las líneas punteadas son a propósito: el scheduler de usuario puede reasignar qué U corre sobre qué K en cualquier momento.

Desde Java 21 (Project Loom), además de los *platform threads* (1:1) existen los **virtual threads**.

Virtual thread

Thread liviano de usuario que se ejecuta montado sobre un **carrier thread** (un platform thread de un pool chico, del orden de la cantidad de cores). Es, ni más ni menos, el modelo N:M.

Cuando un virtual thread hace una operación bloqueante “amigable” (I/O, sockets, etc.), la JVM lo **desmonta** de su carrier thread, deja ese carrier libre para otro virtual thread, y lo vuelve a montar cuando puede seguir. Así se pueden tener millones de virtual threads vivos a la vez, ideal para cargas dominadas por I/O.

Actividades de laboratorio:

- Creación de threads de distintos tipos
- Chequear en **htop** como van apareciendo
- Como inicializar un thread en Java, como apagarlo, simular race conditions

Implementación: Exclusión Mutua

Vimos Dekker, Peterson, Bakery... todos correctos *en el papel*, asumiendo que la única primitiva atómica es leer/escribir una variable compartida.

Llevemos uno de estos algoritmos a un lenguaje real, y corrámoslo en una máquina real.

El corazón de Dekker/Peterson es siempre el mismo patrón: cada thread *escribe su propia bandera* y después *lee la del otro*.

Thread 1

$x = 1$ (A1)

$r1 = y$ (A2)

Thread 2

$y = 1$ (B1)

$r2 = x$ (B2)

Pregunta

Con $x = y = 0$ al empezar, ¿puede terminar $r1 == 0 \ \&\& \ r2 == 0$?

Si pensamos en cualquier *interleaving* de A1,A2,B1,B2 que respete el orden de cada thread:

- Para que $r1==0$: A2 (lee y) tiene que pasar **antes** que B1 ($y=1$). Es decir: $A2 \prec B1$
- Para que $r2==0$: B2 (lee x) tiene que pasar **antes** que A1 ($x=1$). Es decir: $B2 \prec A1$

Pero el orden de programa obliga $A1 \prec A2$ y $B1 \prec B2$. Juntando todo:

$$A1 \prec A2 \prec B1 \prec B2 \prec A1$$

Contradicción

Es un ciclo. En *cualquier* intercalado válido de estas cuatro instrucciones, $r1==0 \ \&\& \ r2==0$ es imposible.

Probemos el patrón anterior en Java de verdad, muchos millones de veces:

```
1 static int x, y, r1, r2;
2
3 // Thread 1                // Thread 2
4 x = 1;                    y = 1;
5 r1 = y;                   r2 = x;
6
7 // ... repetido 20.000.000 de veces, contando cuantas rondas
8 // terminan con (r1 == 0 && r2 == 0)
```

Resultado real (x86)

detections / 20 000 000 rondas violaron consistencia
secuencial

detections **no da 0.**

Acabamos de ver, en una máquina real, exactamente el resultado que **demostramos imposible** dos slides atrás.

La demostración que hicimos asumía una visión (interleaving), es decir, un modelo de un único procesador virtual donde todas las acciones de los hilos se fuerzan a formar un orden total, consistente con el orden de cada programa.

Ese supuesto tiene nombre: **Sequential Consistency (SC)**. Y el hardware real, en general, **no lo cumple**.

Modelo de consistencia (Sorin, Hill & Wood)

Una definición precisa, visible a nivel de arquitectura, de qué es corrección en memoria compartida. Da las reglas que rigen los **loads** y los **stores** (lecturas y escrituras de memoria), y cómo actúan sobre la memoria. (*A Primer on Memory Consistency and Cache Coherence.*)

En criollo: de todos los órdenes posibles en que se podrían intercalar los **loads/stores** de los threads, dice cuáles son **válidos**.

SC es el modelo más fuerte. Hoy vamos a ver varios más débiles.

Intuitivamente, un sistema es *secuencialmente consistente* si se comporta como si hubiera:

- un único procesador,
- ejecutando, de a una por vez, las instrucciones de todos los threads,
- eligiendo en qué orden intercalarlas (el scheduler puede ser adversarial),
- pero **sin** alterar el orden interno de cada thread.

Es exactamente el modelo con el que dibujamos diagramas de transición y trazas al principio de la materia: **un paso a la vez, de un thread a la vez**, mezclados en cualquier orden que respete cada programa.

Sequential Consistency (Lamport, 1979)

Un multiprocesador es secuencialmente consistente si el resultado de cualquier ejecución es el mismo que si las operaciones de **todos** los cores se hubieran ejecutado en algún **orden secuencial**, y las operaciones de cada core aparecen en ese orden **en el orden dado por su programa**.

Para volverlo preciso (y poder compararlo después con modelos más débiles) necesitamos un poquito de notación:

- $L(a), S(a)$: un load/store a la dirección a .
- $\prec_p$: orden de **programa** (el de cada core, por separado).
- $\prec_m$: orden de **memoria** (el orden total, único y global, que estamos buscando).

Con esa notación, SC pide algo muy simple: para **cualquier** par de operaciones consecutivas de un mismo core (a la misma dirección o a direcciones distintas, no importa), el orden de programa se respeta en el orden de memoria.

1 ^{ra} op. \ 2 ^{da} op.	Load(b)	Store(b)
Load(a)	X	X
Store(a)	X	X

Por ejemplo, la casilla Store→Load dice: $S(a) \prec_p L(b) \Rightarrow S(a) \prec_m L(b)$. Las otras tres casillas son análogas. Y a esto se le suma que toda lectura tome el valor de la **última** escritura anterior a ella (misma dirección) según $\prec_m$.

Existen procesadores que lo implementaron al pie de la letra (p.ej. el **MIPS R10000**), pero en general **mantener SC en hardware es caro**.

Ya vimos la tabla: SC marca las cuatro casillas. Bauticémoslas, para poder hablar de modelos *más débiles* que relajan sólo algunas:

- **Load** → **Load (LL)**: dos lecturas pueden verse fuera de orden.
- **Load** → **Store (LS)**: una escritura puede adelantarse a una lectura anterior.
- **Store** → **Store (SS)**: dos escrituras pueden hacerse visibles fuera de orden.
- **Store** → **Load (SL)**: una lectura puede adelantarse a una escritura anterior.

SC = las cuatro casillas marcadas. Un modelo más débil se define, justamente, por **cuáles deja de marcar**. Vamos a dedicarle el resto de la clase a entender de dónde sale una sola de estas cuatro reordenaciones: SL.

Paso 1: ¿por qué puede tardar un store?

Para que un **store** sea **visible**, mi core necesita el permiso de escribir esa línea de memoria: tiene que ser, por un instante, el único que tiene una copia válida de esa dirección. Si otra cache ya la tiene, hay que coordinarse antes: avisarle que la invalide o la ceda, y esperar la confirmación.

De asegurar eso se encarga el **protocolo de coherencia de cache** (p.ej. MESI), con dos grandes familias de implementación: *snoopy* (bus compartido, típico en UMA) o basados en *directorio* (sistemas más grandes, NUMA). Repaso de AyOC, no nos vamos a detener en el detalle.

El costo

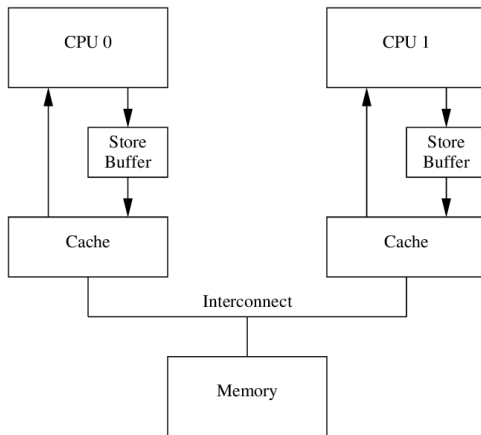
Ese ida y vuelta tiene latencia real, y los stores son **muy frecuentes**: si el core se quedara parado esperando cada uno, tiraríamos a la basura buena parte del beneficio de tener cache.

La solución es **desacoplar** “ejecutar el store” de “el store llega a la memoria compartida”.

Store buffer

Cola **FIFO**, chiquita y **privada de cada core**, ubicada entre el core y la cache/memoria compartida.

- Cuando el core “ejecuta” un **store**, en realidad sólo lo **encola** ahí, con el valor a escribir, y sigue trabajando **sin esperar nada**.
- El store buffer, en paralelo, tramita el permiso de escritura. Cuando lo consigue, recién ahí **drena**: le pasa el valor a la cache compartida, donde otros cores ya lo pueden ver.



Cada CPU tiene su store buffer **propio**, entre ella y su cache. Las caches se coordinan entre sí (coherencia) a través del **interconnect**, y detrás de todo está la memoria principal.

Problema: mientras mi store sigue **pendiente** en el buffer (todavía no llegó a la cache), yo mismo hago un **load** a esa **misma** dirección. Si ese load fuera derecho a la cache, vería el valor **viejo**: rompería la semántica secuencial de mi propio programa.

Store-to-load forwarding

Todo **load** a una dirección α primero chequea **mi propio** store buffer. Si hay un store pendiente a α , uso **ese** valor (el más nuevo en mi orden de programa) en vez de ir a la cache.

Con forwarding, el store buffer es **invisible** para quien sólo mira ese thread: mi propio orden siempre se ve respetado.

Forwarding sólo actúa si el **load** es a la **misma** dirección que tengo bufferizada.
¿Y un load a **otra** dirección?

Ese load **no tiene nada que forwardear**: va directo a la cache, **sin esperar** a que mi store (todavía pendiente en el buffer) drene primero.

El problema

Mi store y ese load posterior, que en mi programa están en **ese** orden, pueden llegar a la memoria compartida en el orden **contrario**. Para cualquier otro core que esté mirando, mi load “pasó” antes que mi store. Eso es, exactamente, **store→load**.

El store buffer es **FIFO** (no reordena internamente), y salvo por forwarding, todo lo demás sigue yendo a la cache en orden de programa: **$S \rightarrow S$, $L \rightarrow L$ y $L \rightarrow S$ se preservan**. Sólo **$S \rightarrow L$ se rompe**: es exactamente el problema del Paso 4.

Eso, con nombre y apellido, es lo que implementan Intel y AMD:

TSO (Total Store Order)

Prohíbe LL, LS y SS. **Permite SL**: una lectura puede completarse antes de que una escritura anterior (a otra dirección) se haga visible a los demás núcleos. Se llama “Total **Store** Order” porque los stores sí quedan totalmente ordenados entre sí, gracias al FIFO.

Con esto en mano, volvamos a **DekkerLitmus** y armemos la traza completa del fallo.

1. Core 0: $x=1 \rightarrow$ va al Store Buffer 0 (todavía **no** está en la cache compartida).
2. Core 1: $y=1 \rightarrow$ va al Store Buffer 1 (ídem, todavía no visible).
3. Core 0: $r1=y \rightarrow$ no tengo y bufferizado, voy a la cache: todavía dice $y=0$.
4. Core 1: $r2=x \rightarrow$ no tengo x bufferizado, voy a la cache: todavía dice $x=0$.
5. *Recién ahora* ambos store buffers drenan: la cache pasa a $x=1$, $y=1$, pero ya es demasiado tarde.

Resultado

$r1 == 0 \ \&\& \ r2 == 0$: cada core **ya vio** su propio store (vía forwarding, si hiciera falta) pero ninguno vio el store del otro a tiempo.

TSO sólo relaja **una** cosa: $S \rightarrow L$. Y encima sabemos **exactamente** de dónde sale: un store que queda esperando en el buffer mientras un load a otra dirección lo pasa de largo.

Entonces, para arreglarlo en un punto puntual del programa, alcanza con una única herramienta que le diga al core: **“antes de seguir de acá, asegurate de que no quede nada pendiente atrás”**.

La pregunta

¿Cómo se le pide eso, concretamente, al hardware?

FENCE (a.k.a. memory barrier)

Instrucción que ordena: toda operación **anterior** al FENCE en orden de programa queda antes, en $\prec_m$, que **cualquier** operación posterior al FENCE.

¿Cómo se implementa? De la forma más directa posible, con el vocabulario que ya tenemos:

- Al ejecutar el FENCE, el core **fuera el drenaje** de su store buffer (espera a que quede vacío).
- No deja ejecutar **ningún** load ni store posterior al FENCE hasta que ese drenaje termine.

En x86 esto es la instrucción `mfence` (o, indirectamente, cualquier instrucción con prefijo `lock`). Como TSO sólo tiene una reordenación para tapar, los FENCE se necesitan poco y son relativamente baratos.

Thread 1

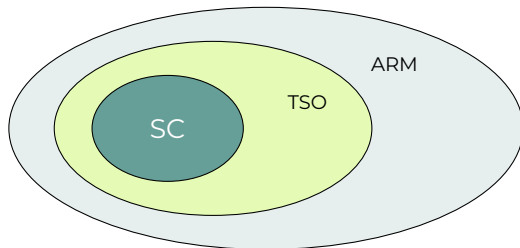
$x = 1$
FENCE
 $r1 = y$

Thread 2

$y = 1$
FENCE
 $r2 = x$

Ahora $r1=y$ no puede ejecutar hasta que el store a x ya haya drenado (y lo mismo del otro lado): $r1==0 \ \&\& \ r2==0$ vuelve a ser **imposible**.

Decimos que un modelo Y es **más relajado** (débil) que X si toda ejecución válida bajo X también es válida bajo Y, pero no al revés.



Toda ejecución SC es también una ejecución TSO válida (nunca al revés: nuestro $(0,0)$ es TSO pero no SC).

x86-TSO es, dentro de los modelos débiles, uno de los **más fuertes**: sólo relaja SL. ARM y POWER (y el default de RISC-V) van mucho más lejos.

La regla general de un modelo relajado

Por **default**, no se ordena nada entre direcciones distintas (ni LL, ni LS, ni SS, ni SL). Lo único gratis es el orden entre accesos de un mismo thread a la **misma** dirección. Todo lo demás: **si lo querés ordenado, lo tenés que pedir**, con una barrera de memoria explícita.

Es exactamente la idea de TSO (“pedile a un FENCE lo que el hardware no te da gratis”), llevada al extremo: acá casi todo hay que pedirlo.

- **POWER:** `sync/hwsync` (pesado, ordena **todo**, incluso $S \rightarrow L$) y `lwsync` (liviano, ordena LL, LS y SS, pero **no** $S \rightarrow L$).
- **ARM:** `dmb` (barrera general). ARMv8 suma anotaciones más livianas de *acquire/release* sobre loads y stores individuales.

Cuanto más débil el modelo, más fences hacen falta, y más importa elegir el **más liviano que alcance** (usar `hwsync` en todos lados también funcionaría, ¡pero se regala toda la performance que viniste a buscar!).

Volvamos a la pregunta incómoda del principio: Dekker, Peterson y Bakery, escritos tal cual, **no andan** en hardware real. Y no queremos que cada programador tenga que razonar sobre store buffers y reordenamientos cada vez que toca una variable compartida.

La salida no es “prohibir la optimización”, es acotar **cuándo** importa:

Idea clave

Si el programa está bien sincronizado, ¿podemos **recuperar** la ilusión de Sequential Consistency, gratis?

FENCE nos da **orden**. Pero para un spinlock hace falta algo más: que un **load+store** (un *read-modify-write*) se ejecuten como un bloque **indivisible**, sin que nadie se meta en el medio.

Es una primitiva distinta, la que ya vimos en pseudocódigo en la teórica: **test-and-set**, **compare-and-swap**, **fetch-and-add**...

¿Cómo son, de verdad, en assembler?

```
1 spin:
2     mov    eax, 1
3     xchg   eax, [lock]    % xchg siempre es atomico, sin
                           lock explicito
4     test   eax, eax
5     jnz    spin           % si daba 1, ya estaba tomado
6     % seccion critica
7
```

`xchg` intercambia, **atómicamente**, un registro con una posición de memoria. Es el **test-and-set** de la teórica, convertido en una instrucción real de hardware.

```
1 retry:
2     mov    eax, esperado
3     mov    ebx, nuevo
4     lock cmpxchg [addr], ebx
5     jnz    retry
6
```

Compara `[addr]` con `eax`: si son iguales, escribe `ebx` (atómicamente) y listo. Si no, trae el valor **real** a `eax` y hay que reintentar. Es el **compare-and-swap** de la teoría: la primitiva más versátil, con la que se arman casi todos los algoritmos *lock-free*.

Con el vocabulario de coherencia que ya tenemos: una instrucción `lock` (y `xchg`, que siempre lo es) hace que el core consiga la línea en estado **exclusivo** y **no la suelte** hasta terminar todo el read-modify-write.

La consecuencia

Ningún otro core puede leer ni escribir esa dirección mientras tanto: el load y el store del RMW quedan **consecutivos** en el orden de memoria, como si nada se hubiera colado en el medio.

Bonus: en x86, una instrucción `lock` también actúa como **FENCE** completo. Por eso muchas implementaciones usan un `lock xadd` sobre un valor descartable en vez de `mfence` para conseguir orden.

En vez de bloquear la línea (*pesimista*), ARM usa *load-linked/store-conditional*: LDREX pone una **reserva** sobre la dirección. STREX sólo tiene éxito si nadie la tocó desde entonces.

```
1 retry:
2     ldrex r0, [addr]      % leo y reservo
3     % ... calculo el nuevo valor en r1
4     strex r2, r1, [addr] % intento escribir, r2=0 si tuvo
    exito
5     cmp    r2, #0
6     bne    retry          % alguien se metio: reintento
7
```

Si STREX falla, no hubo ningún dato corrupto: simplemente hay que **reintentar**. Apostás a que nadie interfiere, en vez de bloquearlos a todos de entrada.

Hasta ahora hablamos del hardware. Pero nosotros no solemos programar en assembler: programamos en Java, C++, Go...

¿Por qué no alcanza con el modelo del hardware?

Porque el **compilador** también reordena, cachea en registros, elimina lecturas “redundantes”, saca invariantes de loops...asumiendo que el código es **secuencial**. Todo eso es válido en un solo thread, y puede romper un programa concurrente **incluso si el hardware fuera perfectamente SC**.

Ejemplos de optimizaciones normales (y correctas, en secuencial) que afectan lo que vimos:

- **Registros:** guardar una variable en un registro y no releerla de memoria (el otro thread puede escribirla y yo nunca me entero).
- **Reordenamiento de instrucciones** sin dependencias aparentes entre sí (para el compilador, x e y son variables no relacionadas).
- **Hoisting / loop-invariant code motion:** sacar una lectura fuera de un loop porque “no cambia”.

Por eso un lenguaje necesita su **propio** modelo de consistencia: un contrato, independiente de la arquitectura destino, que le diga al compilador (y al hardware, vía las instrucciones que el compilador emite) **qué reordenamientos están prohibidos** y le diga al programador **qué tiene que pedir explícitamente** para evitarlos.

Java define su propio contrato: el **Java Memory Model** (JMM, Capítulo 17 del JLS). La revisión que está vigente hoy es de **JSR-133 (2004)**, formalizada por Manson, Pugh y Adve.

La garantía de fondo

Si el programa está “correctamente sincronizado”, se comporta como SC, en cualquier JVM y cualquier arquitectura. Vamos a formalizar esto con el vocabulario **propio** de la JMM.

Conflicto y carrera de datos

Dos operaciones de **datos** conflictúan si son de threads distintos, acceden la **misma** dirección, y al menos una es un **store**. Si conflictúan y no hay, entre medio, un par de operaciones de **sincronización** (una de cada thread) que las ordene, forman una **carrera**. Un programa es **DRF** (*Data-Race-Free*) si, pensándolo bajo SC, **ninguna** de sus ejecuciones tiene una carrera.

Garantía DRF $\Rightarrow$ SC

Si un programa es DRF, el modelo (relajado o no) garantiza que **todas** sus ejecuciones son ejecuciones SC.

Java cumple formalmente con la garantía **DRF** $\Rightarrow$ **SC** utilizando su modelo de memoria (*Java Memory Model* o JMM).

- **Sin carreras (DRF):** Si un programa Java está correctamente sincronizado (es decir, no tiene carreras de datos).
- **Comportamiento SC:** el JMM garantiza que se comportará exactamente como si se ejecutara bajo **Consistencia Secuencial**, ocultando todas las optimizaciones relajadas del hardware.

Para lograrlo, Java define formalmente cuándo existe la sincronización necesaria mediante la relación **happens-before**.

Si una acción A happens-before de una acción B , entonces los resultados de la acción A son garantizados de ser visibles para la acción B , y A se ejecuta lógicamente antes que B .

Con DRF $\Rightarrow$ SC, un programador tiene dos caminos:

- razonar directamente con las reglas del modelo relajado, o
- agregar la sincronización que haga falta y razonar **siempre** con SC, el modelo simple.

Lo mejor es la segunda casi siempre. La primera queda para quien escribe librerías de sincronización.

Dekker, Peterson y Bakery **necesitan** que dos threads escriban y lean la bandera/turno del otro **sin** ningún lock (¡construir exclusión mutua a partir de read/write es justamente el desafío!).

Esas variables de coordinación **conflictúan** entre threads y nada las sincroniza: por definición, **no son DRF**. La garantía SC for DRF no aplica, y el hardware queda libre de reordenar. `DekkerLitmus.java` muestra ese problema con lupa: sin `volatile`, `x` e `y` son exactamente ese tipo de variable.

Garantía Happens-Before

Una escritura en una variable `volatile` ***happens-before*** de cualquier lectura posterior de esa misma variable.

Efecto: Garantiza visibilidad inmediata y evita reordenamientos de instrucciones alrededor de los accesos a la variable, ayudando a cumplir con la condición **DRF**
⇒ **SC**.

```
volatile int x = 0;
volatile int y = 0;
// Thread 1
x = 1;           // Store(x)
// FENCE implicito por volatile
int r1 = y;      // Load(y)

// Thread 2
y = 1;           // Store(y)
// FENCE implicito por volatile
int r2 = x;      // Load(x)
```

-  Daniel J. Sorin, Mark D. Hill, y David A. Wood.
A Primer on Memory Consistency and Cache Coherence.
Synthesis Lectures on Computer Architecture, Morgan & Claypool
Publishers, 2011.